МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

ЛАБОРАТОРНА РОБОТА №2

з курсу: „ Методи та системи штучного інтелекту ”

виконав студент 4го курсу
групи ДА-11
Сидько Н.С.

КИЇВ
2024

Мета роботи: Розглянути архітектуру, основні принципи та етапи реалізації класичної експертної системи. Дослідити ефективність механізму прямого висновку для експертної системи планувального або діагностичного типу.

Завдання:

1. Згідно варіанту (Табл.4.1) сформулювати базу знань ЕС.
2. Сформулювати і формалізувати закінчений набір правил ЕС, що розробляється. Перевірити правила на повноту і несуперечність.
3. Сформувати блок пояснень до експертної системи, що дозволяє в режимі консультації експертної системи проглянути окрему його гілку аж до заданої контрольної точки.
4. Проаналізувати ефективність механізму прямого пошуку для обраної діагностичної задачі.

Варіант завдання

13. Профорієнтація школяра.

Перелік використовуваних апаратних і програмних засобів

Апаратний засіб

Характеристики устройства

Имя устройства	WIN-6UTALTB07GO
Процессор	12th Gen Intel(R) Core(TM) i5-12450H 2.00 GHz

Програмні засоби

Для виконання данного завдання ми використали мову програмування Python для реалізації основної логіки та графічного інтерфейсу, бібліотека tkinter для створення інтерактивного графічного інтерфейсу користувача, а також стандартна структура даних Python (dict) для зберігання дерева рішень, яке виступає в ролі бази знань системи. Усі компоненти інтегровані в межах одного проєкту, що забезпечує простоту та ефективність реалізації експертної системи.

Для побудови схеми дерева було використанно Draw.io через свою простоту

Опис методів і алгоритмів рішення;

1) Ініціалізація системи

У файлі main.py створюється об’єкт класу ExpertSystemGUI, який є основою для взаємодії з користувачем через графічний інтерфейс.

У конструкторі класу (__init__) задаються початкові змінні: history (для збереження відповідей користувача), weights (для накопичення ваг кожної відповіді), result (кінцевий результат) та current_node (поточний вузол у дереві рішень).

2) Відображення питань

Метод show_question отримує вузол з дерева рішень (tree.py) відповідно до значення current_node. Вузол містить запитання, варіанти відповідей із вагами та вказівки на наступний вузол або результат. Якщо вузол є кінцевим (має ключ result), викликається метод show_result.

Для відображення питань створюється текстове поле (Label) для запитання та радіо-кнопки (Radiobutton) для вибору відповіді. У випадку питання про зарплату використовується текстове поле (Entry) для вводу числового значення.

3) Обробка відповідей

Метод enable_next_button активує кнопку "Далі", якщо користувач зробив вибір або ввів значення.

Після натискання кнопки "Далі" викликається метод next_question, який:

1. Додає вибір користувача та відповідну вагу до списків history та weights.
2. Оновлює вузол дерева (current_node), якщо це не кінцевий вузол.
3. Викликає show_question для переходу до наступного запитання або show_result, якщо знайдено результат.

4) Відображення результату

Метод show_result_window створює нове вікно, де відображається результат, пояснення (перелік питань і відповідей) та загальна вага. Для завершення роботи програми використовується кнопка "Закрити", яка викликає метод quit_program.

Логіка роботи ЕС.

Користувач послідовно відповідає на запитання, які формуються на основі попередніх відповідей. Наприклад, якщо користувач обирає, що йому більше подобаються технічні науки, система переходить до уточнюючих запитань про предмети, які цікавлять користувача (наприклад, математика, фізика, інформатика). На кожному етапі відповіді користувача зберігаються в історії разом із вагами, які використовуються для визначення кінцевого результату. Кожне запитання в дереві рішень задається послідовно, а користувач має можливість обрати відповідь із варіантів або ввести числове значення (наприклад, для запитання про бажану зарплату). Відповіді обробляються системою, і на основі дерева рішень визначається наступне запитання або одразу результат. У кінці, після завершення опитування, користувач отримує рекомендацію щодо професії разом із поясненням вибору та загальною вагою відповідей.

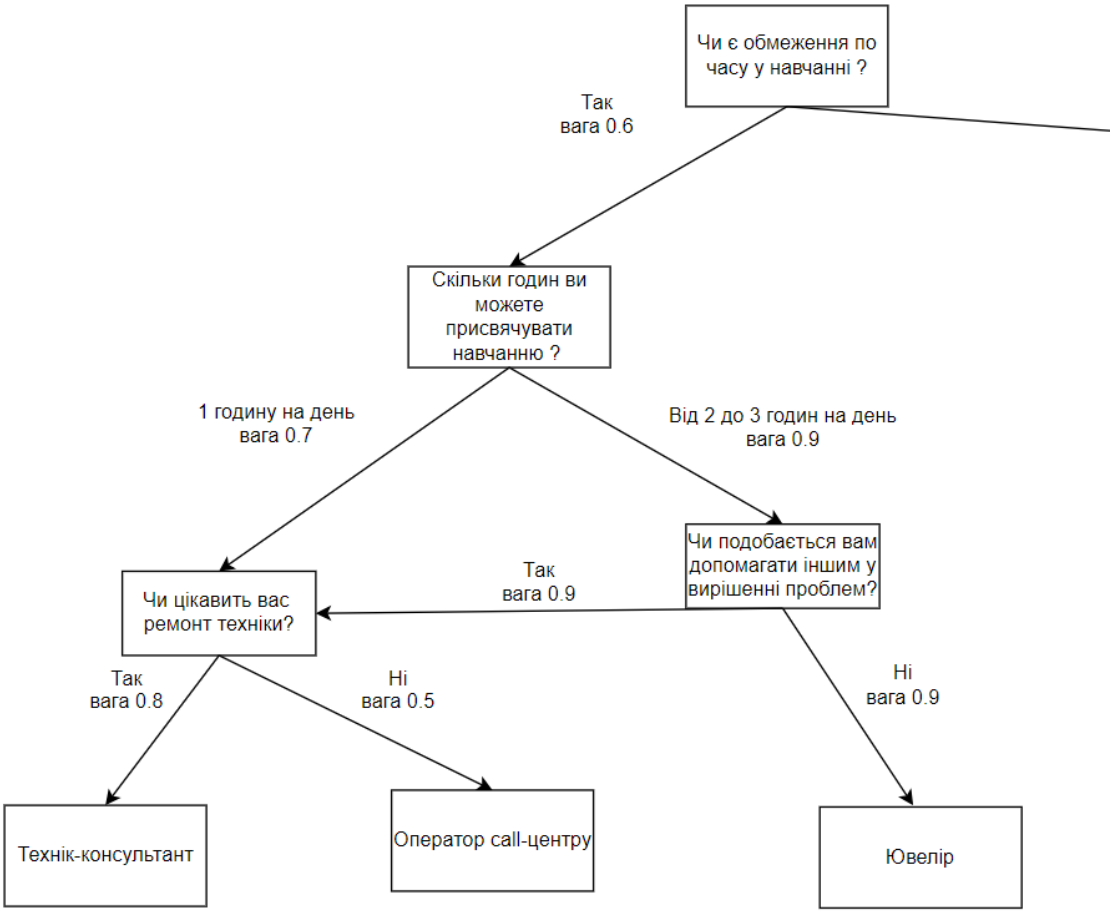
Механізм висновку

У системі використовується прямий механізм висновку (від запитань до результату), де висновок формується шляхом послідовного переходу між

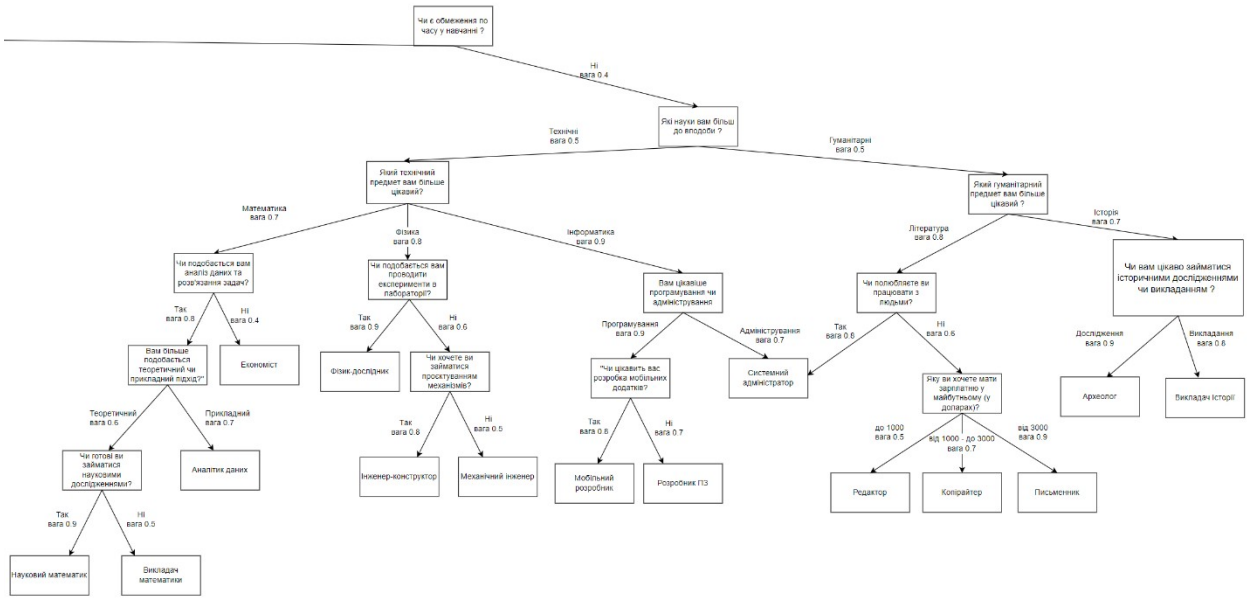
вузлами дерева рішень, починаючи від кореня ("root"). Система задає запитання та на основі відповідей користувача переходить по гілках дерева до кінцевого вузла, який містить професію.

Дерево рішень

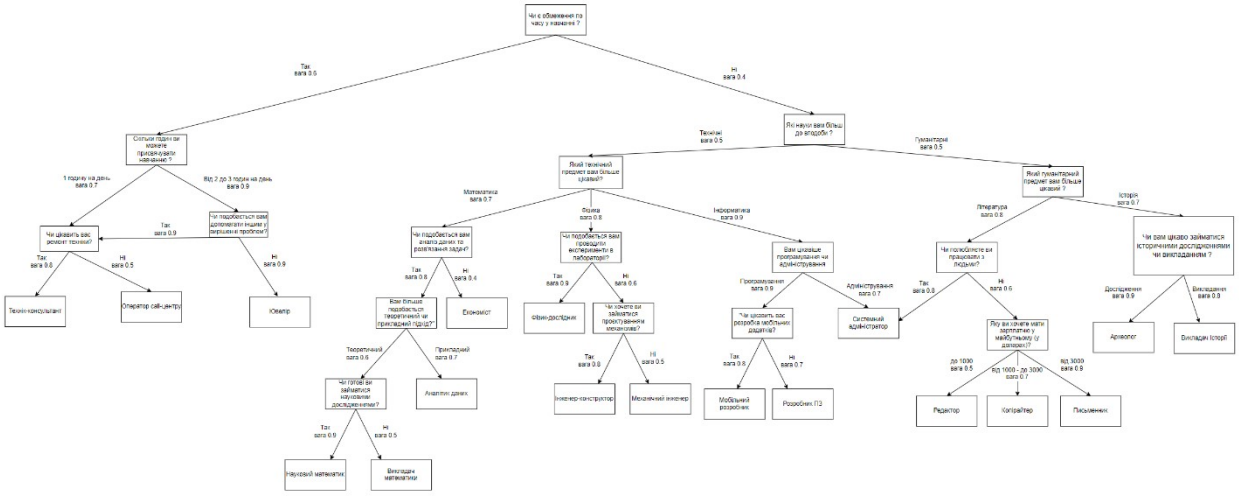
Частина 1



Частина 2



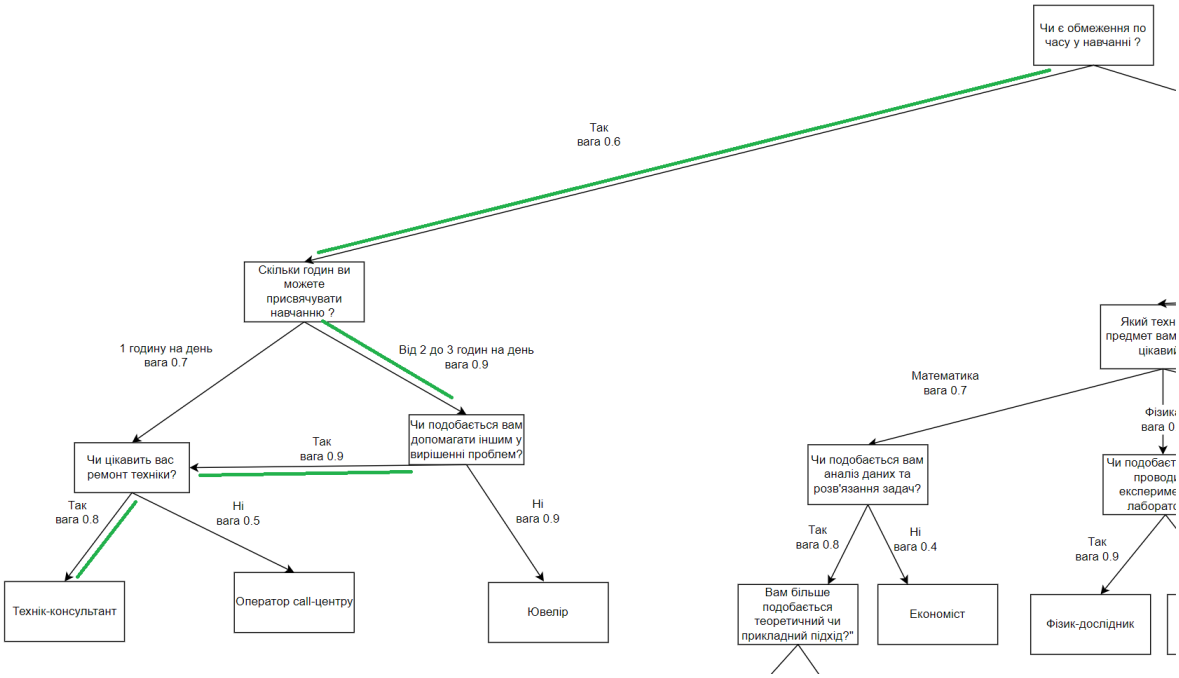
Повне дерево



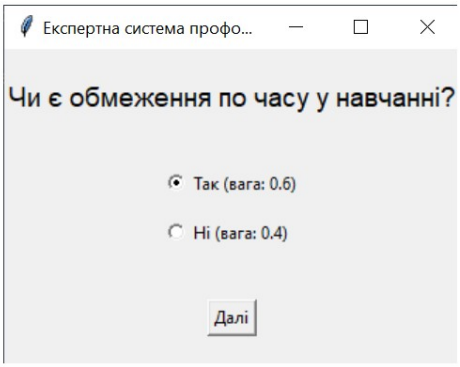
Приклад роботи коду

Розглянемо наступні гілки :

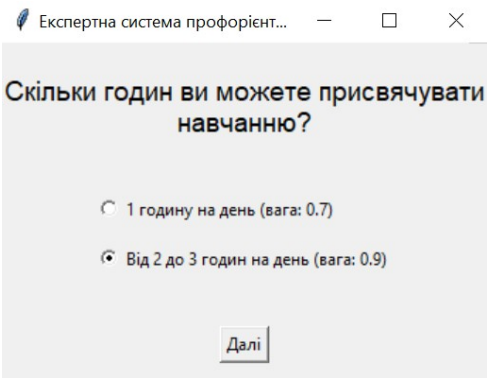
Гілка 1:



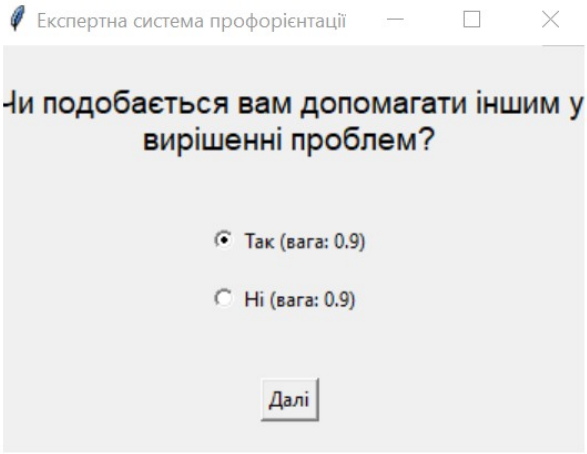
Питання 1



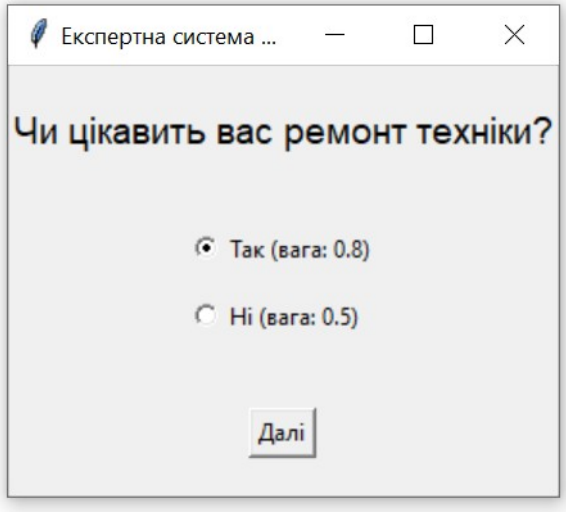
Питання 2



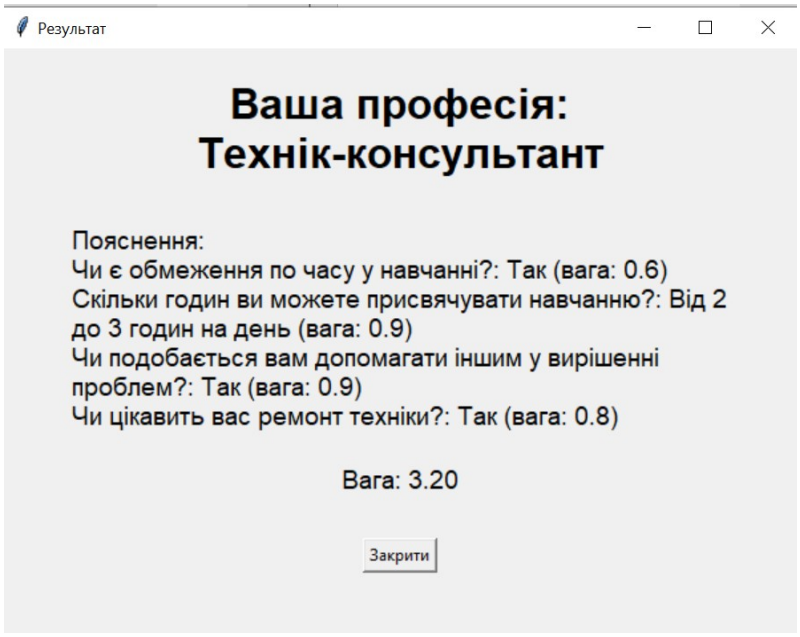
Питання 3



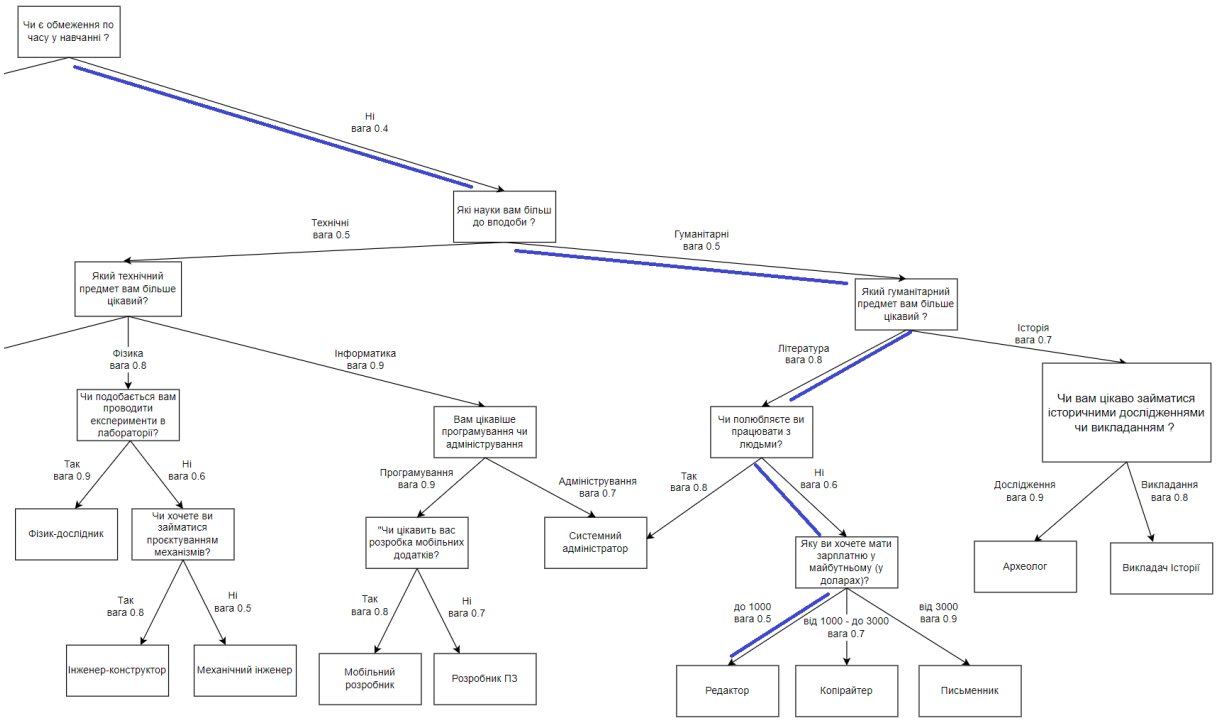
Питання 4



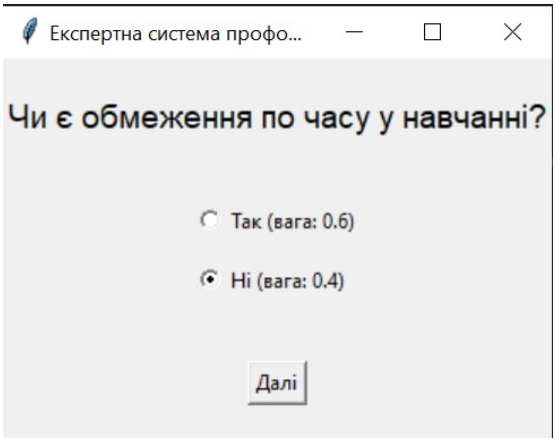
Відповідь



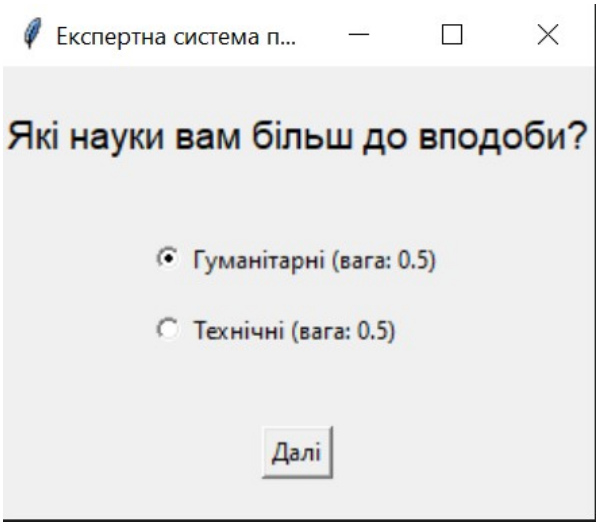
Гілка 2



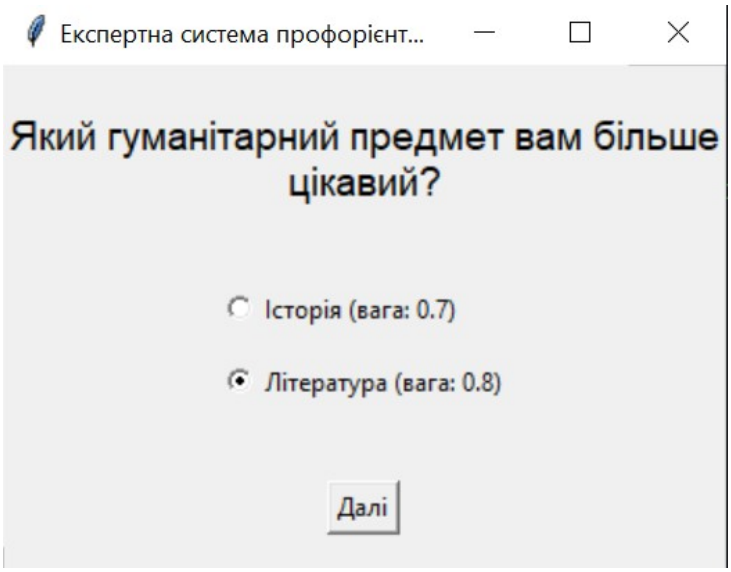
Питання 1



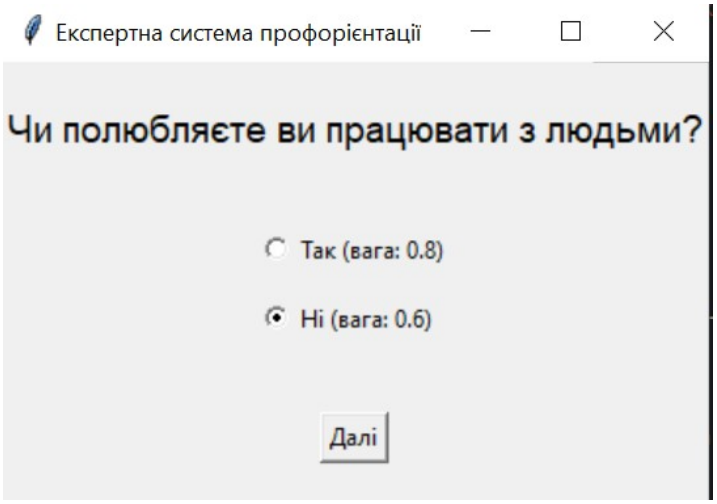
Питання 2



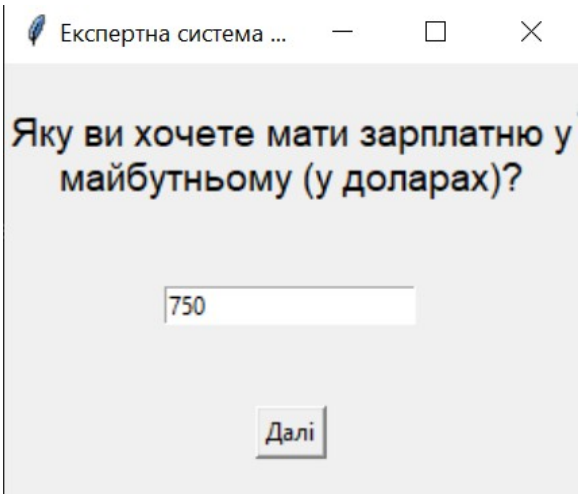
Питання 3



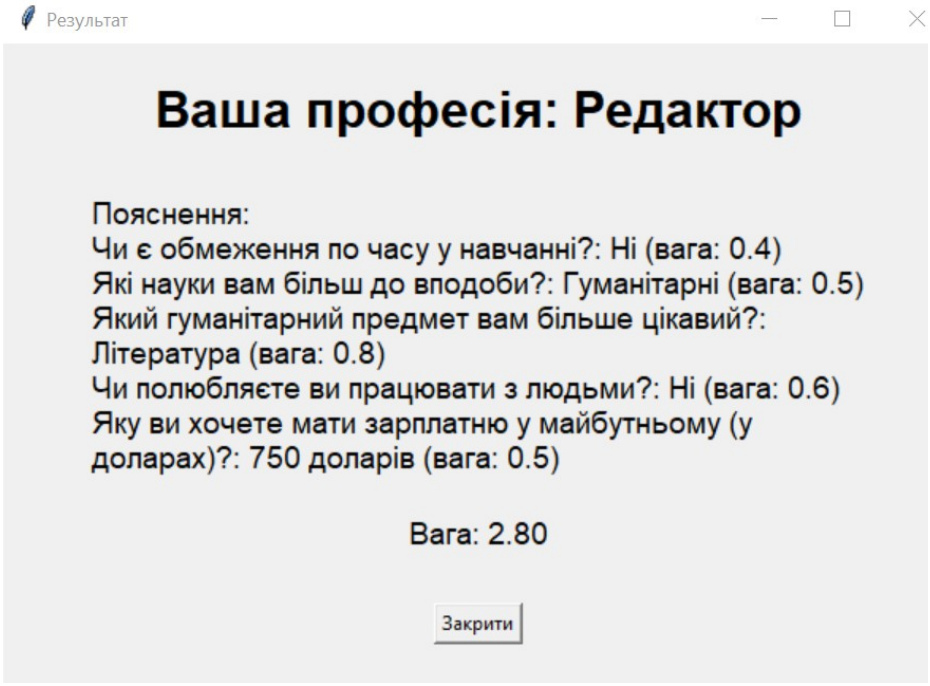
Питання 4



Питання 5



Відповідь



Висновок

У ході виконання лабораторної роботи №2, було розроблено експертну систему з вибору профорієнтації. Було побудоване дерево виборів. Система реалізована на мові програмування Python із використанням бібліотеки Tkinter для створення інтерфейсу. Для реалізації бд було написано словник, який імітує бд. Інтерфейс користувача реалізований таким чином що після відповіді на якесь питання ми йдемо по гілках дерева, підбираючи професію згідно певних критеріїв.

Лістинг коду

Main.py

```
import tkinter as tk
from tkinter import messagebox
from tree import decision_tree

class ExpertSystemGUI:
    def __init__(self, root):
        self.root = root
        self.root.title("Експертна система профорієнтації")
        self.history = []
        self.weights = []
        self.result = ""
        self.current_node = "root"
        self.selected_answer = None


        self.question_label = tk.Label(self.root, text="", font=("Arial", 14), wraplength=400)
        self.question_label.pack(pady=20)

        self.options_frame = tk.Frame(self.root)
        self.options_frame.pack(pady=10)

        self.next_button = tk.Button(self.root, text="Далі", command=self.next_question,
state=tk.DISABLED)
```

```
        self.next_button.pack(pady=20)

        self.show_question()

    def show_question(self):
        for widget in self.options_frame.winfo_children():
            widget.destroy()

        node = decision_tree.get(self.current_node)
        if not node:
            self.show_result()
            return

        self.question_label.config(text=node["question"])
        self.selected_answer = None
        self.next_button.config(state=tk.DISABLED)


        if node["question"] == "Яку ви хочете мати зарплатню у майбутньому (у доларах)?":
            self.salary_entry = tk.Entry(self.options_frame)
            self.salary_entry.pack(pady=10)
            self.salary_entry.bind("<KeyRelease>", self.enable_next_button)
        else:

            self.answer_var = tk.StringVar(value="init")
            for answer, data in node["answers"].items():
                button = tk.Radiobutton(
                    self.options_frame,
                    text=f"{answer} (вага: {data['weight']})",
                    variable=self.answer_var,
                    value=answer,
                    command=self.enable_next_button
                )
                button.pack(anchor="w", pady=5)

    def enable_next_button(self, event=None):
        """Активує кнопку 'Далі', коли користувач робить вибір або вводить значення."""
        if self.answer_var.get() != "init" or (hasattr(self, "salary_entry") and self.salary_entry.get().isdigit()):
            self.next_button.config(state=tk.NORMAL)

    def next_question(self):
        """Обробляє перехід до наступного питання після натискання кнопки 'Далі'."""
        node = decision_tree[self.current_node]


        if node["question"] == "Яку ви хочете мати зарплатню у майбутньому (у доларах)?":
            salary_input = self.salary_entry.get()


            if not salary_input.isdigit():
                messagebox.showerror("Помилка вводу", "Будь ласка, введіть коректне число для
зарплати.")
                return

            try:
                salary = int(salary_input)
            except ValueError:
                messagebox.showerror("Помилка вводу", "Неможливо обробити введене значення.
Введіть число.")
                return
```

```
            if salary > 10_000_000:
                messagebox.showerror("Помилка вводу", "Введене значення занадто велике. Введіть
реалістичну зарплату.")
                return


            self.history.append((node["question"], f"{salary} доларів"))
            if salary <= 1000:
                self.weights.append(0.5)
                self.result = "Редактор"
            elif 1000 < salary <= 3000:
                self.weights.append(0.7)
                self.result = "Копірайтер"
            else:
                self.weights.append(0.9)
                self.result = "Письменник"
            self.show_result()
            return


        selected_option = self.answer_var.get()
        if selected_option == "init":
            messagebox.showwarning("Попередження", "Будь ласка, оберіть відповідь.")
            return

        self.history.append((node["question"], selected_option))
        self.weights.append(node["answers"][selected_option].get("weight", 0))

        if "next" in node["answers"][selected_option]:
            self.current_node = node["answers"][selected_option]["next"]
            self.show_question()
        elif "result" in node["answers"][selected_option]:
            self.result = node["answers"][selected_option]["result"]
            self.show_result()

    def show_result_window(self):
        """Відображає нове вікно з результатами та завершує програму після закриття."""
        if not self.result:
            self.result = "Немає результату"


        total_weight = sum(self.weights)


        result_window = tk.Toplevel(self.root)
        result_window.title("Результат")
        result_window.geometry("600×500")


        result_label = tk.Label(
            result_window,
            text=f"Ваша професія: {self.result}",
            font=("Arial", 24, "bold"),
            wraplength=500,
            justify="center"
        )
        result_label.pack(pady=20)


        explanations_text = "\n".join([f"{q}: {a} (вага: {w})" for (q, a), w in zip(self.history, self.weights)])
        explanations_label = tk.Label(
            result_window,
```

```
            text=f"Пояснення:\n{explanations_text}",
            font=("Arial", 14),
            wraplength=500,
            justify="left"
        )
        explanations_label.pack(pady=10)


        total_weight_label = tk.Label(
            result_window,
            text=f"Вага: {total_weight:.2f}",
            font=("Arial", 14),
            wraplength=500,
            justify="center"
        )
        total_weight_label.pack(pady=10)


        close_button = tk.Button(result_window, text="Закрити", command=self.quit_program)
        close_button.pack(pady=20)

    def quit_program(self):
        """Закриває програму повністю."""
        self.root.quit()
        self.root.destroy()

    def show_result(self):
        """Відображає результат користувачеві."""
        self.show_result_window()

if __name__ == "__main__":
    root = tk.Tk()
    app = ExpertSystemGUI(root)
    root.mainloop()
```

Tree.py

```
decision_tree = {
    "root": {
        "question": "Чи є обмеження по часу у навчанні?",
        "answers": {
            "Так": {"weight": 0.6, "next": "q1"},
            "Ні": {"weight": 0.4, "next": "q2"}
        }
    },

    "q1": {
        "question": "Скільки годин ви можете присвячувати навчанню?",
        "answers": {
            "1 годину на день": {"weight": 0.7, "next": "q3"},
            "Від 2 до 3 годин на день": {"weight": 0.9, "next": "q4"}
        }
    },
    "q3": {
        "question": "Чи цікавить вас ремонт техніки?",
        "answers": {
            "Так": {"weight": 0.8, "result": "Технік-консультант"},
            "Ні": {"weight": 0.5, "result": "Оператор call-центру"}
        }
    },
```

```
"q4": {
  "question": "Чи подобається вам допомагати іншим у вирішенні проблем?",
  "answers": {
    "Так": {"weight": 0.9, "next": "q5"},
    "Ні": {"weight": 0.9, "result": "Ювелір"}
  }
},
"q5": {
  "question": "Чи цікавить вас ремонт техніки?",
  "answers": {
    "Так": {"weight": 0.8, "result": "Технік-консультант"},
    "Ні": {"weight": 0.5, "result": "Оператор call-центру"}
  }
},

"q2": {
  "question": "Які науки вам більш до вподоби?",
  "answers": {
    "Гуманітарні": {"weight": 0.5, "next": "q6"},
    "Технічні": {"weight": 0.5, "next": "q12"}
  }
},

"q6": {
  "question": "Який гуманітарний предмет вам більше цікавий?",
  "answers": {
    "Історія": {"weight": 0.7, "next": "q7"},
    "Література": {"weight": 0.8, "next": "q8"}
  }
},
"q7": {
  "question": "Чи вам цікаво займатися історичними дослідженнями чи викладанням?",
  "answers": {
    "Дослідження": {"weight": 0.9, "result": "Археолог"},
    "Викладання": {"weight": 0.8, "result": "Викладач Історії"}
  }
},
"q8": {
  "question": "Чи полюбляєте ви працювати з людьми?",
  "answers": {
    "Так": {"weight": 0.8, "result": "Системний адміністратор"},
    "Ні": {"weight": 0.6, "next": "q9"}
  }
},
"q9": {
  "question": "Яку ви хочете мати зарплатню у майбутньому (у доларах)?",
  "answers": {
    "1000": {"weight": 0.5, "result": "Редактор"},
    "1500-3000": {"weight": 0.7, "result": "Копірайтер"},
    "понад 5000": {"weight": 0.9, "result": "Письменник"}
  }
},

"q12": {
  "question": "Який технічний предмет вам більше цікавий?",
  "answers": {
    "Інформатика": {"weight": 0.9, "next": "q13"},
    "Фізика": {"weight": 0.8, "next": "q15"},
    "Математика": {"weight": 0.7, "next": "q18"}
  }
},
"q13": {
```

```
    "question": "Вам цікавіше програмування чи адміністрування?",
    "answers": {
      "Адміністрування": {"weight": 0.7, "result": "Системний адміністратор"},
      "Програмування": {"weight": 0.9, "next": "q14"}
    }
  },
  "q14": {
    "question": "Чи цікавить вас розробка мобільних додатків?",
    "answers": {
      "Так": {"weight": 0.8, "result": "Мобільний розробник"},
      "Ні": {"weight": 0.7, "result": "Розробник ПЗ"}
    }
  },
  "q15": {
    "question": "Чи подобається вам проводити експерименти в лабораторії?",
    "answers": {
      "Так": {"weight": 0.9, "result": "Фізик-дослідник"},
      "Ні": {"weight": 0.6, "next": "q16"}
    }
  },
  "q16": {
    "question": "Чи хочете ви займатися проєктуванням механізмів?",
    "answers": {
      "Так": {"weight": 0.8, "result": "Інженер-конструктор"},
      "Ні": {"weight": 0.5, "result": "Механічний інженер"}
    }
  },
  "q18": {
    "question": "Чи подобається вам аналіз даних та розв'язання задач?",
    "answers": {
      "Так": {"weight": 0.8, "next": "q19"},
      "Ні": {"weight": 0.4, "result": "Економіст"}
    }
  },
  "q19": {
    "question": "Вам більше подобається теоретичний чи прикладний підхід?",
    "answers": {
      "Прикладний": {"weight": 0.7, "result": "Аналітик даних"},
      "Теоретичний": {"weight": 0.6, "next": "q20"}
    }
  },
  "q20": {
    "question": "Чи готові ви займатися науковими дослідженнями?",
    "answers": {
      "Так": {"weight": 0.9, "result": "Науковий математик"},
      "Ні": {"weight": 0.5, "result": "Викладач математики"}
    }
  }
}
```